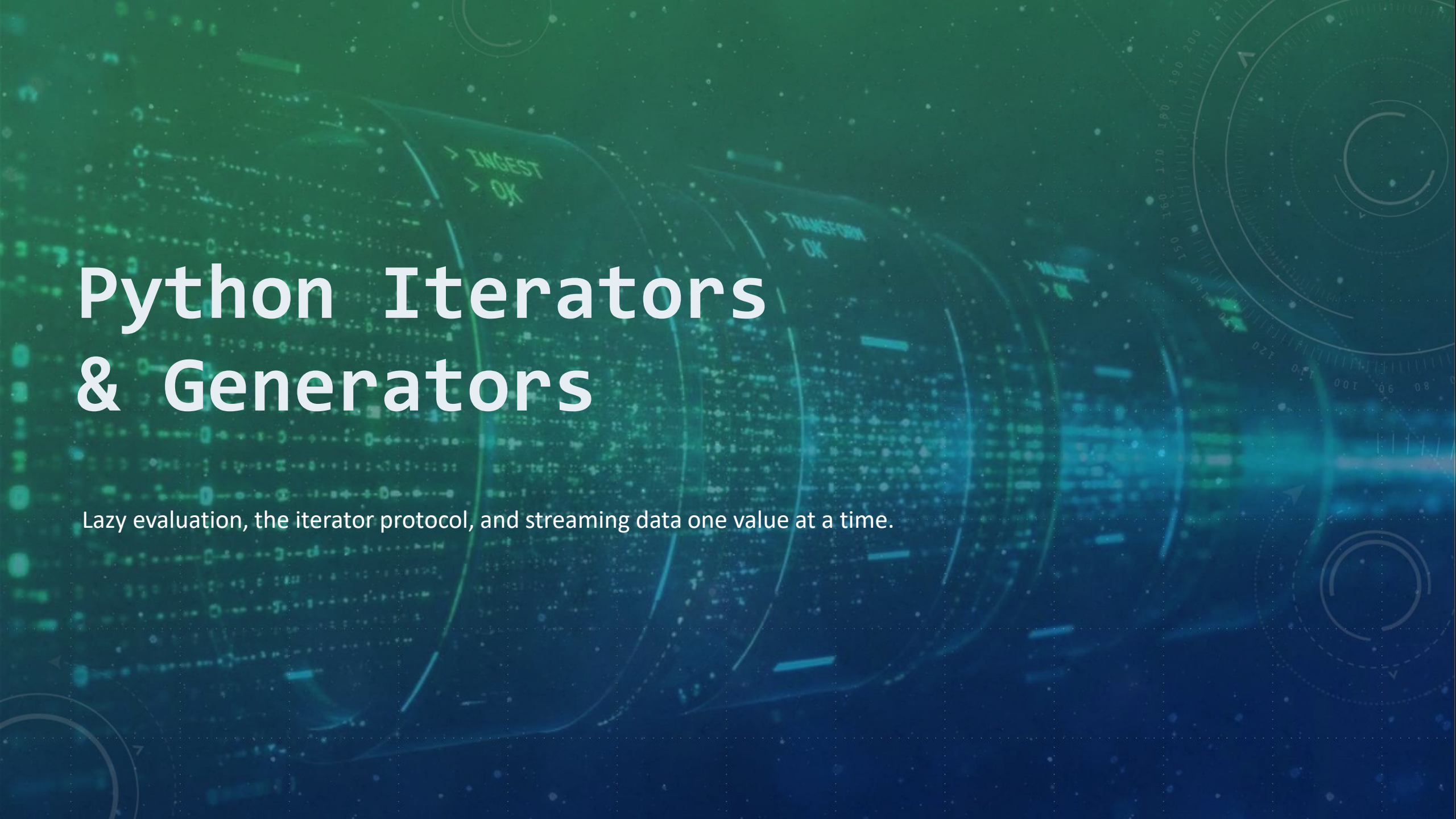


Python Iterators & Generators

Lazy evaluation, the iterator protocol, and streaming data one value at a time.

Iterables vs. iterators

ITERABLE

Anything you can loop over.

Implements `__iter__()`, which returns a fresh iterator.

`lists` · `tuples` · `dicts` · `sets` · `strings` · `files`

ITERATOR

The object doing the work.

Implements `__next__()` to yield the next value, and returns `self` from __iter__.

`Stateful` · `single-use` · `exhausts once`

`iter()` turns an iterable into an iterator; `next()` pulls one value at a time.

```
>>> nums = [10, 20, 30]          # an iterable
>>> it = iter(nums)   # an iterator   >>> next(it)  -> 10
```

The iterator protocol

`__iter__(self)`

Returns the iterator object itself.

`__next__(self)`

Returns the next value in the sequence.

`raise StopIteration`

Signals there are no more values.

count_up.py

```
class CountUp:
    def __init__(self, n):
        self.n = n
        self.i = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        self.i += 1
        return self.i

for n in CountUp(3): print(n)  # 1 2 3
```

How a for-loop really works

what you write

```
for item in data:
    process(item)
```

what Python does

```
it = iter(data)          # get iterator
while True:
    try:
        item = next(it)  # pull one
    except StopIteration:
        break            # done
    process(item)
```

iter() grab an iterator » **next()** ask for a value » **yield / return** use the value » **StopIteration** loop exits cleanly

Generators: iterators, minus the boilerplate

A **generator function** uses `yield` instead of `return`.

- **Calling it returns a generator**
  No code runs yet — you get an object.
- **yield pauses & saves state**
  Execution freezes; locals stay alive.
- **next() resumes where it left off**
  Runs until the next yield.
- **__iter__ / __next__ are free**
  Python writes the protocol for you.

countdown.py

```
def countdown(n):
    while n > 0:
        yield n          # pause here
        n -= 1

g = countdown(3)     # nothing runs
next(g)   -> 3
next(g)   -> 2
next(g)   -> 1
next(g)   -> StopIteration

# 13 lines of class
# collapsed into 4.
```

yield pauses — and remembers

fibonacci.py

```python
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

from itertools import islice
seq = fib()
list(islice(seq, 7))
# -> [0, 1, 1, 2, 3, 5, 8]
```

Local state survives every pause:

call 1	a=0	b=1	yields 0
call 2	a=1	b=1	yields 1
call 3	a=1	b=2	yields 1
call 4	a=2	b=3	yields 2
call 5	a=3	b=5	yields 3

Lazy evaluation: compute only what you need

~1
value in memory at a time

∞
sequences you can model

0
up-front compute cost

eager — builds the whole list

```
# 10 million ints in RAM
nums = []
for x in range(10**7):
    nums.append(x*x)
total = sum(nums)

# ~400 MB allocated  ✗
```

lazy — streams one value

```
def squares(n):
    for x in range(n): yield x*x
total = sum(squares(10**7))

# ~constant memory  ✓
```

>_ side by side

Class-based iterator vs. generator

Aspect	Class iterator	Generator
Boilerplate	`__iter__`+ `__next__`+ state	one def with yield
State handling	manual, in instance attrs	automatic, in the frame
StopIteration	you raise it explicitly	raised on function return
Readability	verbose	concise & linear
When to reach for it	complex, reusable objects	most everyday cases

Where generators replace iterators

Streaming big files

Read a line at a time — never load a multi-GB log into RAM.

```
for line in open(path):
```

Infinite sequences

Counters, IDs, sensor feeds — model streams with no end.

```
while True: yield n; n += 1
```

Pipelines

Chain generators so each stage pulls from the last, lazily.

```
sum(map(square, stream))
```

Paged API results

Yield each record; fetch the next page only when asked.

```
yield from fetch(page)
```

Exception handling: the anatomy

try:

Wrap the code that might fail.

except:

Catch a specific exception type.

else:

Runs only if no exception fired.

finally:

Always runs — ideal for cleanup.

commit.py

```python
try:
    value = risky()
except ValueError:
    value = fallback()   # handle it
except (KeyError, IndexError) as e:
    log(e)
    raise                # re-raise
else:
    commit(value)        # ran clean
finally:
    cleanup()            # always runs
```

Raise, re-raise & define your own

config.py

```python
class ConfigError(Exception):
    """Raised on bad config."""

def load(cfg):
    if "port" not in cfg:
        raise ConfigError("missing port")
    return cfg["port"]

try:
    load({})
except ConfigError as e:
    print(f"bad config: {e}")
```

Catch narrow

Prefer specific types over a bare except:.

Preserve context

Chain causes with raise New() from err.

Subclass Exception

Not BaseException; that catches Ctrl-C.

Never silence

Avoid except: pass — it hides real bugs.

>_ iterator ties

Exceptions drive iteration

stop.py

```
it = iter([1, 2])
next(it)           # -> 1
next(it)           # -> 2
next(it)           # StopIteration
next(it, "done")    # -> 'done'
```

StopIteration is a signal, not a bug.
The for-loop catches it internally to know when to stop — it's control flow.

Give next() a default

next(it, fallback) returns the default instead of raising when the iterator is spent.

PEP 479 — no leaks

A StopIteration that escapes a generator body becomes a RuntimeError, so it can't silently end a loop.

Cleanup & control in generators

read_lines.py

```
def read_lines(path):
    f = open(path)
    try:
        for line in f:
            yield line
    finally:
        f.close()   # guaranteed
```

control.py

```
g = read_lines("big.log")
next(g)           # opens, yields

g.close()          # -> GeneratorExit
                     # finally closes f

g.throw(ValueError)  # inject error
```

try/finally always cleans up

Resources close even if the consumer stops early or breaks.

Let GeneratorExit propagate

Don't catch and swallow it on .close(); re-raise or return.

Best practices & pitfalls

✓ DO

- Prefer generators for large or streaming data.
- Remember: an iterator is exhausted after one full pass.
- Use yield from to delegate to a sub-generator.
- Reach for itertools (islice, chain, count) before rolling your own.

✗ AVOID

- Re-looping a spent generator; it yields nothing the second time.
- Calling len() on a generator; it has no length.
- Building a huge list when a lazy stream would do.
- Forgetting a generator holds resources open (e.g. files) until consumed.

>_ recap

Key takeaways

1. **Iterable ≠ iterator**
   Iterables make iterators via iter(); iterators produce values via next().

2. **The protocol is small**
   __iter__, __next__, and StopIteration; that's the whole contract.

3. **Generators do it for free**
   yield turns a function into an iterator and manages state for you.

4. **Lazy by default**
   Values are computed on demand; constant memory, even for infinite streams.

```
>_
one
value
at a time,

just in
time.
```